

CSCE 411-501 ✖ Spring 2026

Undecidability Practice

Friday, January 23, 2026

1. Prove that the following languages are undecidable:

(a) $\text{HALTS-EMPTY} := \{\langle P \rangle : P(\varepsilon) \text{ halts}\}$

Solution: To prove that HALTS-EMPTY is undecidable, we show that $\text{HALTS} \leq_T \text{HALTS-EMPTY}$. Let HEDECIDER be a decider for HALTS-EMPTY . We create the following decider HALTSDECIDER for HALTS :

$\text{HALTSDECIDER}(P, x)$

Write out the source code of the following algorithm HELPER :

$\text{HELPER}(y)$

Simulate $P(x)$

Return True

Return $\text{HEDECIDER}(\text{HELPER})$

Correctness:

- Suppose $\langle P, x \rangle \in \text{HALTS}$. Then $P(x)$ halts, so HELPER halts on all inputs, including ε . Thus, $\langle \text{HELPER} \rangle \in \text{HALTS-EMPTY}$, so $\text{HEDECIDER}(\text{HELPER})$ will return True, so $\text{HALTSDECIDER}(P, x)$ will return True.
- Suppose $\langle P, x \rangle \notin \text{HALTS}$. Then $P(x)$ loops, so HELPER loops forever on every input, including ε . Thus $\langle \text{HELPER} \rangle \notin \text{HALTS-EMPTY}$, so $\text{HEDECIDER}(\text{HELPER})$ will return False, so $\text{HALTSDECIDER}(P, x)$ will return False.

(b) $\text{ALMOST-EMPTY} := \{\langle P \rangle : P \text{ is an algorithm and } |L(P)| \leq 411\}$

Solution: To prove that ALMOST-EMPTY is undecidable, we show that $\text{HALTS} \leq_T \text{ALMOST-EMPTY}$. Let AEDECIDER be a decider for ALMOST-EMPTY . We create the following decider HALTSDECIDER for HALTS :

$\text{HALTSDECIDER}(P, x)$

Write out the source code of the following algorithm HELPER :

$\text{HELPER}(y)$

Simulate $P(x)$

Return True

Return $\neg \text{AEDECIDER}(\text{HELPER})$

Correctness:

- Suppose $\langle P, x \rangle \in \text{HALTS}$. Then $P(x)$ halts, so $L(\text{HELPER}) = \Sigma^*$, and thus $|L(\text{HELPER})| = \infty > 411$. Therefore $\langle \text{HELPER} \rangle \notin \text{ALMOST-EMPTY}$, so $\text{AEDECIDER}(\text{HELPER})$ will return False, so $\text{HALTSDECIDER}(P, x)$ will return True.
- Suppose $\langle P, x \rangle \notin \text{HALTS}$. Then $P(x)$ loops, so $L(\text{HELPER}) = \emptyset$, and thus $|L(\text{HELPER})| = 0 \leq 411$. Therefore $\langle \text{HELPER} \rangle \in \text{ALMOST-EMPTY}$, so $\text{AEDECIDER}(\text{HELPER})$ will return True, so $\text{HALTSDECIDER}(P, x)$ will return False.

(c) $\text{COMPLEMENT} := \{\langle P_1, P_2 \rangle : P_1, P_2 \text{ are algorithms \& } L(P_1) = \Sigma^* \setminus L(P_2)\}$

Solution: To prove that COMPLEMENT is undecidable, we show that $\text{HALTS} \leq_T \text{COMPLEMENT}$. Let COMPLEMENTDECIDER be a decider for COMPLEMENT . We create the following decider HALTSDECIDER for HALTS :

$\text{HALTSDECIDER}(P, x)$

Write out the source code of the following algorithm HELPER1 :

$\text{HELPER1}(y)$

Simulate $P(x)$

Return True

Write out the source code of the following algorithm HELPER2 :

$\text{HELPER2}(y)$

Return False

Return $\text{COMPLEMENTDECIDER}(\text{HELPER1}, \text{HELPER2})$

Correctness: We note that since HELPER2 returns False on all inputs, $L(\text{HELPER2}) = \emptyset$ and $\Sigma^* \setminus L(\text{HELPER2}) = \Sigma^*$, whether or not $P(x)$ halts.

- Suppose $\langle P, x \rangle \in \text{HALTS}$. Then $P(x)$ halts, so $L(\text{HELPER1}) = \Sigma^*$, and thus $L(\text{HELPER1}) = \Sigma^* \setminus L(\text{HELPER2})$. Thus $\langle \text{HELPER1}, \text{HELPER2} \rangle \in \text{COMPLEMENT}$, so $\text{COMPLEMENTDECIDER}(\text{HELPER1}, \text{HELPER2})$ will return True, so $\text{HALTSDECIDER}(P, x)$ will return True.
- Suppose $\langle P, x \rangle \notin \text{HALTS}$. Then $P(x)$ loops, so $L(\text{HELPER1}) = \emptyset$, and thus $L(\text{HELPER1}) \neq \Sigma^* \setminus L(\text{HELPER2})$. Therefore $\langle \text{HELPER1}, \text{HELPER2} \rangle \notin \text{COMPLEMENT}$, so $\text{COMPLEMENTDECIDER}(\text{HELPER1}, \text{HELPER2})$ will return False, so $\text{HALTSDECIDER}(P, x)$ will return False.

(d) $\text{EVEN-DECIDER} := \{\langle P \rangle : P \text{ is a decider for } \{\langle n \rangle : n \text{ is an even integer}\}\}$

Solution: Let

$$\text{EVEN} := \{ \langle n \rangle : n \text{ is an even integer} \}$$

To prove that EVEN-DECIDER is undecidable, we show that $\text{HALTS} \leq_T \text{EVEN-DECIDER}$. Let EDDECIDER be a decider for EVEN-DECIDER. We create the following decider HALTSDECIDER for HALTS:

HALTSDECIDER(P, x)

Write out the source code of the following algorithm HELPER:

HELPER(n)

Simulate $P(x)$

If $n \bmod 2 = 0$:

Return True

Else:

Return False

Return EDDECIDER(HELPER)

Correctness:

- Suppose $\langle P, x \rangle \in \text{HALTS}$. Then $P(x)$ halts, so HELPER returns True all even integers and returns False on all other inputs. Therefore HELPER decides EVEN. Thus $\langle \text{HELPER} \rangle \in \text{EVEN-DECIDER}$, so EDDECIDER(HELPER) will return True, so HALTSDECIDER(P, x) will return True.
- Suppose $\langle P, x \rangle \notin \text{HALTS}$. Then $P(x)$ loops, so HELPER(n) loops for every integer n , and thus HELPER does not decide EVEN. Therefore $\langle \text{HELPER} \rangle \notin \text{EVEN-DECIDER}$, so EDDECIDER(HELPER) will return False, so HALTSDECIDER(P, x) will return False.

2. Let L be a language with $|L| = 1$. Prove that L is decidable.

Solution: Let s be the single string in L . We give the following decider for L :

$M_L(x)$

If($x = s$):

Return True

Return False

This algorithm returns True all strings in $L = \{s\}$ and returns False all other strings, so it is decider for L .

3. Show that the following language is undecidable:

$$\text{DPCP} := \{ \langle P, d \rangle : P \text{ is an instance of the Post correspondence problem, } d \text{ is a domino in that instance, and } P \text{ has a solution which does not use the domino } d. \}$$

For example, Suppose P is the the following set of dominos,

$$\left\{ \left[\frac{b}{ca} \right], \left[\frac{a}{ab} \right], \left[\frac{bb}{a} \right], \left[\frac{ca}{a} \right], \left[\frac{abc}{c} \right] \right\},$$

Then $\langle P, \left[\frac{bb}{a} \right] \rangle \in \text{DPCP}$, because P has the following solution,

$$\left[\frac{a}{ab} \right] \left[\frac{b}{ca} \right] \left[\frac{ca}{a} \right] \left[\frac{a}{ab} \right] \left[\frac{abc}{c} \right],$$

which does not use the domino

$$\begin{bmatrix} bb \\ \hline a \end{bmatrix}$$

Solution: To prove that DPCP is undecidable, we show that $PCP \leq_T DPCP$. Let DPCP-DECIDER be a decider for DPCP. We create the following decider PCP-DECIDER for PCP:

PCP-DECIDER(P)

$d \leftarrow$ the domino $\begin{bmatrix} \\ \hline a \end{bmatrix}$

$P' \leftarrow P$ with the domino d added

Return DPCP-DECIDER(P', d)

Correctness:

- Suppose $\langle P \rangle \in PCP$. Then P has a solution without does not use the domino d (since d is not part of P), and P' has the same solution (since P' has all the dominoes P has), so DPCP-DECIDER(P', d) will return True, so PCP-DECIDER(P) will return True.
- Suppose $\langle P \rangle \notin PCP$. Since any solution to P' which doesn't use d would also be a solution to P , this means that P' has no solutions that don't use d . Therefore $\langle P, d \rangle \notin DPCP$ so DPCP-DECIDER(P', d) will return False, so PCP-DECIDER(P) will return False.

4. Show that the function $f : \{P \mid P \text{ is an algorithm}\} \times \mathbb{Z}^+ \rightarrow \mathbb{Z}^+$ defined by $f(P, n) := \max_{x \in \Sigma^n \text{ s.t. } P(x) \text{ halts}}$ the number of steps that $P(x)$ runs for before halting. is uncomputable.

Solution: AFSOC that f is computable. Let MAXSTEPS be an algorithm that computes f . We construct the following decider for HALTS:

HALTSDecider(P, x)
 $n \leftarrow |x|$
 $t \leftarrow \text{MAXSTEPS}(P, n)$
Simulate $P(x)$ for t steps
If the simulation of $P(x)$ halted:
 Return True
Return False

We will now prove that HALTSDecider correctly decides HALTS:

- Suppose that $\langle P, x \rangle \in \text{HALTS}$. By definition, $f(P, |x|)$ is the maximum number of steps that P takes to halt across all inputs of length $|x|$ on which P halts, so $P(x)$ must halt in at most $f(P, |x|)$ steps. Therefore, when HALTSDecider simulates P for MAXSTEPS($P, |x|$) steps, P will halt and HALTSDecider(P, x) will return True.
- Suppose that $\langle P, x \rangle \notin \text{HALTS}$. Then $P(x)$ loops forever, so when we simulate it for t steps it will not halt. Thus HALTSDecider(P, x) will return False.

Thus, we have constructed a decider for HALTS, but we know from the course notes that HALTS is undecidable. This is a contradiction, so our initial assumption that f was computable must be false.